In [18]:
```
#This software is a free software. Thus, it is licensed under GNU General Public License.
#Python implementation to Smith-Waterman Algorithm for Homework 1 of Bioinformatics class.
#Forrest Bao, Sept. 26 <http://fsbao.net> <forrest.bao aT gmail.com>
#Taken by Zander Furanas from github user alevchuk on 4/17/2017. Will be used as the starting codebase from which to
 create Semantic Smith-Waterman

# zeros() was origianlly from NumPy.
# This version is implemented by alevchuk 2011-04-10

import nltk
from gensim.models.keyedvectors import KeyedVectors
from nltk.corpus import stopwords

#hardcoded stuff that ultimately shouldn't be
scale = 10
match_award      =  scale
gap_penalty      = - scale/2 # both for opening and extanding
word_vectors = KeyedVectors.load_word2vec_format('../GoogleNews-vectors-negative300.bin', binary=True)
stop = set(stopwords.words('english'))
mismatch_penalty = - scale/2
```

In [31]:
```
# -*- coding: utf-8 -*-
import os
def write_data(data, header, writepath):
    ordered_fieldnames = header
    if os.path.exists(writepath):
        with open(writepath, "a") as dataset:
            writer = csv.writer(dataset, dialect = "excel")
            writer.writerow(data)
    else:
        with open(writepath, "w") as dataset:
            csv.DictWriter(dataset, dialect = "excel", fieldnames = ordered_fieldnames).writeheader()
            writer = csv.writer(dataset, dialect = "excel")
            writer.writerow(data)
```

In [23]:

```
#This software is a free software. Thus, it is licensed under GNU General Public License.
#Python implementation to Smith-Waterman Algorithm for Homework 1 of Bioinformatics class.
#Forrest Bao, Sept. 26 <http://fsbao.net> <forrest.bao aT gmail.com>
#Taken by Zander Furanas from github user alevchuk on 4/17/2017. Will be used as the starting codebase from which to
 create Semantic Smith-Waterman

# zeros() was origianlly from NumPy.
# This version is implemented by alevchuk 2011-04-10
def zeros(shape):
    retval = []
    for x in range(shape[0]):
        retval.append([])
        for y in range(shape[1]):
            retval[-1].append(0)
    return retval



def sw_match_score(alpha, beta):
    if alpha == beta:
        return match_award
    elif alpha == '-' or beta == '-':
        return gap_penalty
    else:
        return mismatch_penalty

def sw_finalize(align1, align2):
    align1 = align1[::-1]    #reverse sequence 1
    align2 = align2[::-1]    #reverse sequence 2

    i,j = 0,0

    #calcuate identity, score and aligned sequeces
    symbol = ''
    found = 0
    score = 0
    identity = 0
    for i in range(0,len(align1)):
        # if two AAs are the same, then output the letter
        if align1[i] == align2[i]:
            symbol = symbol + align1[i]
            identity = identity + 1
            score += sw_match_score(align1[i], align2[i])

        # if they are not identical and none of them is gap
        elif align1[i] != align2[i] and align1[i] != '-' and align2[i] != '-':
            score += sw_match_score(align1[i], align2[i])
```

```
            symbol += ' '
            found = 0

        #if one of them is a gap, output a space
        elif align1[i] == '-' or align2[i] == '-':
            symbol += ' '
            score += gap_penalty

    identity = float(identity) / len(align1) * 100

    #print 'Identity =', "%3.3f" % identity, 'percent'
    #print 'Score =', score
    #print align1
    #print symbol
    #print align2

    return score


def water(seq1, seq2):
    m, n = len(seq1), len(seq2)  # length of two sequences

    # Generate DP table and traceback path pointer matrix
    score = zeros((m+1, n+1))      # the DP table
    pointer = zeros((m+1, n+1))    # to store the traceback path

    max_score = 0        # initial maximum score in DP table
    # Calculate DP table and mark pointers
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            score_diagonal = score[i-1][j-1] + sw_match_score(seq1[i-1], seq2[j-1])
            score_up = score[i][j-1] + gap_penalty
            score_left = score[i-1][j] + gap_penalty
            score[i][j] = max(0,score_left, score_up, score_diagonal)
            if score[i][j] == 0:
                pointer[i][j] = 0 # 0 means end of the path
            if score[i][j] == score_left:
                pointer[i][j] = 1 # 1 means trace up
            if score[i][j] == score_up:
                pointer[i][j] = 2 # 2 means trace left
            if score[i][j] == score_diagonal:
                pointer[i][j] = 3 # 3 means trace diagonal
            if score[i][j] >= max_score:
                max_i = i
                max_j = j
                max_score = score[i][j];

    align1, align2 = [], []    # initial sequences

    i,j = max_i,max_j    # indices of path starting point

    #traceback, follow pointers
    while pointer[i][j] != 0:
        if pointer[i][j] == 3:
            align1.append(seq1[i-1])
            align2.append(seq2[j-1])
            i -= 1
            j -= 1
        elif pointer[i][j] == 2:
            align1.append('-')
            align2.append(seq2[j-1])
            j -= 1
        elif pointer[i][j] == 1:
            align1.append(seq1[i-1])
            align2.append('-')
            i -= 1

    return sw_finalize(align1, align2)


def get_mismatch_penalty(word1, word2):
    try:
        word_sim = word_vectors.similarity(word1.lower(), word2.lower()) -.5
        match_score = word_sim * scale*2
        if match_score < 0:
            match_score = - scale/2
    except KeyError:
        if word1 == word2:
            match_score = scale
        elif word1 in stop and word2 in stop:
            match_score = -float(scale)/2
        else:
            match_score = - scale/2

    return match_score
```

```
def semantic_match_score(alpha, beta):
    if alpha == beta:
        return match_award
    elif alpha == '-' or beta == '-':
        return gap_penalty
    else:
        return get_mismatch_penalty(alpha, beta)

def semantic_finalize(align1, align2):
    align1 = align1[::-1]    #reverse sequence 1
    align2 = align2[::-1]    #reverse sequence 2

    i,j = 0,0

    #calcuate identity, score and aligned sequeces
    symbol = ''
    found = 0
    score = 0
    identity = 0
    for i in range(0,len(align1)):
        # if two AAs are the same, then output the letter
        if align1[i] == align2[i]:
            symbol = symbol + align1[i]
            identity = identity + 1
            score += semantic_match_score(align1[i], align2[i])

        # if they are not identical and none of them is gap
        elif align1[i] != align2[i] and align1[i] != '-' and align2[i] != '-':
            score += semantic_match_score(align1[i], align2[i])
            symbol += ' '
            found = 0

        #if one of them is a gap, output a space
        elif align1[i] == '-' or align2[i] == '-':
            symbol += ' '
            score += gap_penalty

    identity = float(identity) / len(align1) * 100

    #print 'Identity =', "%3.3f" % identity, 'percent'
    #print 'Score =', score
    #print align1
    #print symbol
    #print align2

    return score


def semantic_water(seq1, seq2):
    m, n = len(seq1), len(seq2)  # length of two sequences

    # Generate DP table and traceback path pointer matrix
    score = zeros((m+1, n+1))      # the DP table
    pointer = zeros((m+1, n+1))    # to store the traceback path

    max_score = 0        # initial maximum score in DP table
    # Calculate DP table and mark pointers
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            score_diagonal = score[i-1][j-1] + semantic_match_score(seq1[i-1], seq2[j-1])
            score_up = score[i][j-1] + gap_penalty
            score_left = score[i-1][j] + gap_penalty
            score[i][j] = max(0,score_left, score_up, score_diagonal)
            if score[i][j] == 0:
                pointer[i][j] = 0 # 0 means end of the path
            if score[i][j] == score_left:
                pointer[i][j] = 1 # 1 means trace up
            if score[i][j] == score_up:
                pointer[i][j] = 2 # 2 means trace left
            if score[i][j] == score_diagonal:
                pointer[i][j] = 3 # 3 means trace diagonal
            if score[i][j] >= max_score:
                max_i = i
                max_j = j
                max_score = score[i][j];

    align1, align2 = [], []    # initial sequences

    i,j = max_i,max_j    # indices of path starting point

    #traceback, follow pointers
    while pointer[i][j] != 0:
        if pointer[i][j] == 3:
            align1.append(seq1[i-1])
```

```
            align2.append(seq2[j-1])
            i -= 1
            j -= 1
        elif pointer[i][j] == 2:
            align1.append('-')
            align2.append(seq2[j-1])
            j -= 1
        elif pointer[i][j] == 1:
            align1.append(seq1[i-1])
            align2.append('-')
            i -= 1

    return semantic_finalize(align1, align2)
```

Everything above imports and constructs the standard Smith-Waterman (SW) local alignment algorithm and the Semantic Smith-Waterman (SSW) local alignment algorithm, ready to function on a test corpus. In the document below, I work with a corpus of tweets from members of Congress and their challengers from the 2016 election, to look at differences in text re-use identified by the standard SW and SSW.

I use a simple vector space model to find candidates for comparison to reduce the search space for each case of reuse rather than comparing each tweet to all other tweets in the corpus. Retweets ahve been removed.

In [1]:
```
import logging
logging.basicConfig(format='%(asctime)s : %(levelname)s : %(message)s', level=logging.INFO)
from gensim import corpora, similarities
import nltk
import csv
from nltk.corpus import stopwords

stop = set(stopwords.words('english'))
```

In [2]:
```
documents = []
with open('/Users/alexanderfurnas/Projects/SemanticSmithWaterman/data/congress_tweets.csv', 'rb') as csvfile:
    tweetreader = csv.reader(csvfile, dialect = "excel")
    next(tweetreader)
    for row in tweetreader:
        documents.append(unicode(row[9], errors='replace'))
```

The cell above reads in all the tweet documents (ignoring any metadata for the moment). In the next step, these are processed simply prior to creating a standard vector space.

In [3]:
```
def sent_tok(sent):
    sent_tok_result = [word.lower() for word in nltk.word_tokenize(sent) if word.isalpha()]
    return sent_tok_result

texts = [sent_tok(document) for document in documents]

print("Texts tokenized")

# remove words that appear only once
from collections import defaultdict
frequency = defaultdict(int)
for text in texts:
    for token in text:
        frequency[token] += 1

print("Text frequencies counted")

texts = [[token for token in text if frequency[token] > 1]
         for text in texts]
```

```
Texts tokenized
Text frequencies counted
```

In [4]:
```
dictionary = corpora.Dictionary(texts)
dictionary.save('congresstweets.dict')  # store the dictionary, for future reference
print(dictionary)
```

```
Dictionary(105135 unique tokens: [u'fawn', u'thebarlock', u'repjimcooper', u'tipponline', u'sowell']...)
```

In [5]:
```
corpus = [dictionary.doc2bow(text) for text in texts]
corpora.MmCorpus.serialize('congresstweets.mm', corpus)
```

In [6]:
```
corpus = corpora.MmCorpus('congresstweets.mm')
```

In [7]:
```
index = similarities.Similarity('/tmp/tst', corpus, num_features = corpus.num_terms)
```

In [8]:
```
index.save('congresstweets.index')
```

In [9]:
```
index.num_best = 100
```

In [ ]:
```
for doc_id in range(0, len(corpus)):
    doc = corpus[doc_id]
    doc_text = texts[doc_id]
    for result in index[doc]:
        result_text = texts[result[0]]
        sw_score = water(doc_text, result_text)
        ssw_score = semantic_water(doc_text, result_text)
        results = (doc_id +1, result[0]+1, sw_score, ssw_score)
        write_data(results, ["query_docid","result_docid", "sw_score", "ssw_score"], "../data/tweet_comparisons.csv")
```